Smart Contract Audit Report

Contract: vulnerable_contract

Date: 2025-07-08 23:00:42

Risk Summary

Risk Score: 60/100

Risk Level: High

Vulnerabilities Found: 6

Vulnerability Findings

1. Use of system() call

Severity: Unknown

Location: system()

Description: Use of system() allows arbitrary command execution. Avoid using it in smart contracts.

2. Unchecked arithmetic operation

Severity: Unknown

Location: balance += amount

Description: Operation 'balance += amount' might cause overflow/underflow. Consider using safe math checks.

3. Unchecked arithmetic operation

Severity: Unknown

Location: balance -= amount

Description: Operation 'balance -= amount' might cause overflow/underflow. Consider using safe math checks.

4. Unchecked arithmetic operation

Severity: Unknown

Location: char* input

Description: Operation 'char* input' might cause overflow/underflow. Consider using safe math checks.

5. Unchecked arithmetic operation

Severity: Unknown

Location: a * b

Description: Operation 'a * b' might cause overflow/underflow. Consider using safe math checks.

6. Unchecked arithmetic operation

Severity: Unknown

Location: rm -rf

Description: Operation 'rm -rf' might cause overflow/underflow. Consider using safe math checks.

AI-Powered Analysis

Audit Summary: Vulnerable Contract

The vulnerable_contract smart contract has been identified to contain five vulnerabilities, including a critical severity issue, three medium severity issues, and one false positive. This summary provides an in-depth analysis of each vulnerability, including severity, impact, potential exploit paths, and mitigation strategies.

1. Use of system() call (High Severity)

* **Description:** The contract uses the `system()` function, which allows arbitrary command execution. This is a critical security risk, as it enables an attacker to execute malicious system commands.
* **Impact:** An attacker can exploit this vulnerability to execute arbitrary system commands, potentially leading to unauthorized access, data tampering, or even complete system compromise.
* **Exploit Path:** An attacker can inject malicious input to the `system()` function, allowing them to execute arbitrary commands.
* **Mitigation Strategy:** Avoid using the `system()` function in smart contracts. Instead, use contract-specific functionality or libraries that provide secure alternatives for executing system-level operations.

Advice for Developers: Never use the `system()` function in smart contracts, as it poses a significant security risk. Instead, focus on using contract-specific functionality or libraries that provide secure alternatives.

2. Unchecked arithmetic operation (Medium Severity)

* **Description:** The contract performs unchecked arithmetic operations, such as `balance += amount`, `balance -= amount`, and `a * b`. These operations can cause overflow or underflow, leading to unexpected behavior.

* **Impact:** An attacker can exploit these vulnerabilities to manipulate the contract's state, potentially leading to unauthorized access or asset theft.
* **Exploit Path:** An attacker can craft malicious input to cause overflow or underflow, allowing them to manipulate the contract's state.
* **Mitigation Strategy:** Use safe math libraries or implement custom safe math checks to prevent overflow and underflow. Ensure that arithmetic operations are properly validated and sanitized.

Advice for Developers: Always use safe math libraries or implement custom safe math checks to prevent overflow and underflow. Validate and sanitize arithmetic operations to ensure the contract's state remains consistent and secure.

3. Unchecked arithmetic operation (Medium Severity)

* **Description:** The contract performs unchecked arithmetic operation `char* input`. This operation can cause overflow or underflow, leading to unexpected behavior.
* **Impact:** An attacker can exploit this vulnerability to manipulate the contract's state, potentially leading to unauthorized access or asset theft.
* **Exploit Path:** An attacker can craft malicious input to cause overflow or underflow, allowing them to manipulate the contract's state.
* **Mitigation Strategy:** Use safe math libraries or implement custom safe math checks to prevent overflow and underflow. Ensure that arithmetic operations are properly validated and sanitized.

Advice for Developers: Always use safe math libraries or implement custom safe math checks to prevent overflow and underflow. Validate and sanitize arithmetic operations to ensure the contract's state remains consistent and secure.

4. Unchecked arithmetic operation (False Positive)

* **Description:** The contract performs an operation `rm -rf`. This is not an arithmetic operation and is likely a mistake in the audit report.
* **Impact:** None, as this is a false positive.
* **Exploit Path:** None, as this is a false positive.
* **Mitigation Strategy:** None, as this is a false positive.

Advice for Developers: Ignore this finding, as it is a false positive. Ensure that the audit report is accurate and reliable to avoid unnecessary confusion.

In conclusion, the vulnerable_contract smart contract contains critical and medium severity vulnerabilities that require immediate attention. Developers should prioritize mitigating these vulnerabilities by avoiding the use of `system()` calls, implementing safe math checks, and validating arithmetic operations. By following these recommendations, the contract can be secured, and the risk of exploitation can be significantly reduced.